

Ejercicios: trazar especificaciones
Unidad 0: Herramientas y preliminares

Eduardo Acuña Yeomans

Lenguajes de Programación
Semestre 2026-2

Lic. en Ciencias de la Computación
Universidad de Sonora

Sesión 6

Esta hoja no se entrega y no se califica, pero es la que más conviene hacer completa: el formato que se practica aquí es exactamente el que se pide en la sección 3 del ejercicio integrador.

Al final está la solución de cada trazado, entera. Compárala paso por paso, no solo el resultado final: lo que se está practicando es el procedimiento.

Las tres reglas del formato

1. Cada paso reemplaza **una** llamada por el lado derecho de la regla que le toca.
2. Al lado de cada paso se anota **qué regla** se aplicó. Una sola regla por renglón: no se juntan dos reglas **distintas** en el mismo paso.
3. Las sustituciones se escriben. No se hacen de cabeza.

Bloque 1: funciones definidas por casos

Empezamos en terreno conocido: dos funciones sobre números, cada una con sus dos reglas.

1. Sea h definida así:

$$\text{[base]} \quad h(0) = 2 \qquad \text{[rec]} \quad h(n) = h(n-1) + 3 \quad \text{para } n > 0$$

Traza $h(4)$. Anota qué regla usas en cada paso.

2. Sea g definida así:

$$\text{[base]} \quad g(0) = 0 \qquad \text{[rec]} \quad g(n) = n + g(n-1) \quad \text{para } n > 0$$

Traza $g(3)$.

3. **Sin trazar** ¿Cuántas veces se aplica la regla [rec] al trazar $h(10)$? ¿Y la regla [base]? Contéstalo sin escribir el trazado.

Bloque 2: calc

Esta es la especificación que ya trabajaron, con **neg**:

[num]	(calc (num-exp n))	= n
[add]	(calc (add-exp e_1 e_2))	= (calc e_1) + (calc e_2)
[mul]	(calc (mul-exp e_1 e_2))	= (calc e_1) $\times$ (calc e_2)
[neg]	(calc (neg-exp e))	= -(calc e)

4. Traza `add(2, mul(3, 4))`.
5. Traza `mul(add(1, 1), add(2, 3))`.
6. Traza `neg(add(2, 3))`.

Sugerencia: por dónde se empieza

Escribe primero el árbol completo con `num-exp`, `add-exp` y demás. El trazado se hace sobre el árbol, no sobre el texto. Si escribes (`calc add(2, mul(3, 4))`) te vas a atorar, porque ninguna regla tiene esa forma del lado izquierdo.

7. Comprobar Teclea las tres expresiones en el REPL y compara con el resultado de tus trazados. Si alguno no coincide, encuentra el paso donde se rompe.

Bloque 3: una especificación nueva

Esta es la especificación de `cuantos-nodos`, la función que escribiste en la hoja anterior. No la has trazado nunca: aplica el mismo formato a reglas que no habías visto.

[n-num]	(cuantos-nodos (num-exp n))	= 1
[n-add]	(cuantos-nodos (add-exp e_1 e_2))	= 1 + (cuantos-nodos e_1) + (cuantos-nodos e_2)
[n-mul]	(cuantos-nodos (mul-exp e_1 e_2))	= 1 + (cuantos-nodos e_1) + (cuantos-nodos e_2)
[n-neg]	(cuantos-nodos (neg-exp e))	= 1 + (cuantos-nodos e)

8. Traza `cuantos-nodos` sobre `add(3, mul(2, 4))`.
9. Traza `cuantos-nodos` sobre `neg(add(1, 2))`.
10. **Sin trazar** Compara la regla `[add]` de `calc` con la regla `[n-add]` de `cuantos-nodos`. ¿En qué se parecen y en qué se diferencian? ¿Por qué las dos funciones se pueden trazar exactamente con el mismo procedimiento, aunque una devuelva el valor de una expresión y la otra cuente nodos?

Soluciones

Clave de Respuesta: bloque 1

Ejercicio 1. $h(4)$:

$$\begin{aligned} h(4) &= h(3) + 3 && [\text{rec}] \\ &= (h(2) + 3) + 3 && [\text{rec}] \\ &= ((h(1) + 3) + 3) + 3 && [\text{rec}] \\ &= (((h(0) + 3) + 3) + 3) + 3 && [\text{rec}] \\ &= (((2 + 3) + 3) + 3) + 3 && [\text{base}] \\ &= 14 \end{aligned}$$

Los paréntesis no son adorno: dicen qué se calculó primero. El último renglón no cita ninguna regla porque es aritmética, no es de la definición de h .

Ejercicio 2. $g(3)$:

$$\begin{aligned} g(3) &= 3 + g(2) && [\text{rec}] \\ &= 3 + (2 + g(1)) && [\text{rec}] \\ &= 3 + (2 + (1 + g(0))) && [\text{rec}] \\ &= 3 + (2 + (1 + 0)) && [\text{base}] \\ &= 6 \end{aligned}$$

Ejercicio 3. La regla [rec] se aplica **10 veces** (una por cada descenso, de 10 hasta 1) y la [base] **una sola vez**. Siempre es así: el caso base se usa exactamente una vez, y es lo que hace que el trazado termine.

Clave de Respuesta: bloque 2: calc

Ejercicio 4. `add(2, mul(3, 4))`:

```
(calc (add-exp (num-exp 2) (mul-exp (num-exp 3) (num-exp 4))))  
= (calc (num-exp 2)) + (calc (mul-exp (num-exp 3) (num-exp 4))) [add]  
= 2 + (calc (mul-exp (num-exp 3) (num-exp 4))) [num]  
= 2 + ((calc (num-exp 3)) × (calc (num-exp 4))) [mul]  
= 2 + (3 × (calc (num-exp 4))) [num]  
= 2 + (3 × 4) [num]  
= 2 + 12  
= 14
```

Ejercicio 5. `mul(add(1, 1), add(2, 3))`. Abreviamos `D = (add-exp (num-exp 2) (num-exp 3))` para que quepa:

```
(calc (mul-exp (add-exp (num-exp 1) (num-exp 1)) D))  
= (calc (add-exp (num-exp 1) (num-exp 1))) × (calc D) [mul]  
= ((calc (num-exp 1)) + (calc (num-exp 1))) × (calc D) [add]  
= (1 + (calc (num-exp 1))) × (calc D) [num]  
= (1 + 1) × (calc D) [num]  
= 2 × (calc D)  
= 2 × ((calc (num-exp 2)) + (calc (num-exp 3))) [add]  
= 2 × (2 + (calc (num-exp 3))) [num]  
= 2 × (2 + 3) [num]  
= 2 × 5  
= 10
```

Abreviar con una letra está permitido, siempre que digas qué es, como aquí con `D`. Vas a volver a necesitarlo más adelante, cuando los trazados crezcan.

Ejercicio 6. `neg(add(2, 3))`:

```
(calc (neg-exp (add-exp (num-exp 2) (num-exp 3))))  
= -(calc (add-exp (num-exp 2) (num-exp 3))) [neg]  
= -((calc (num-exp 2)) + (calc (num-exp 3))) [add]  
= -(2 + (calc (num-exp 3))) [num]  
= -(2 + 3) [num]  
= -5
```

Clave de Respuesta: bloque 3: cuantos-nodos

Ejercicio 8. cuantos-nodos sobre $\text{add}(3, \text{mul}(2, 4))$:

```
(cuantos-nodos (add-exp (num-exp 3) (mul-exp (num-exp 2) (num-exp 4))))  
= 1 + (cuantos-nodos (num-exp 3))                                [n-add]  
  + (cuantos-nodos (mul-exp (num-exp 2) (num-exp 4)))  
= 1 + 1 + (cuantos-nodos (mul-exp (num-exp 2) (num-exp 4)))      [n-num]  
= 1 + 1 + (1 + (cuantos-nodos (num-exp 2)) + (cuantos-nodos (num-exp 4))) [n-mul]  
= 1 + 1 + (1 + 1 + (cuantos-nodos (num-exp 4)))                [n-num]  
= 1 + 1 + (1 + 1 + 1)                                           [n-num]  
= 5
```

Ejercicio 9. cuantos-nodos sobre $\text{neg}(\text{add}(1, 2))$:

```
(cuantos-nodos (neg-exp (add-exp (num-exp 1) (num-exp 2))))  
= 1 + (cuantos-nodos (add-exp (num-exp 1) (num-exp 2)))          [n-neg]  
= 1 + (1 + (cuantos-nodos (num-exp 1)) + (cuantos-nodos (num-exp 2))) [n-add]  
= 1 + (1 + 1 + (cuantos-nodos (num-exp 2)))                      [n-num]  
= 1 + (1 + 1 + 1)                                                 [n-num]  
= 4
```

Ejercicio 10. Las dos reglas tienen la misma **forma**: a la izquierda, una llamada sobre $(\text{add-exp } e_1 \ e_2)$, y a la derecha, algo armado con las llamadas sobre e_1 y e_2 . Lo que cambia es con qué se combinan los resultados de las partes. Una los suma tal cual, la otra los suma y agrega 1 por el nodo en el que está parada.

Se trazan igual porque el procedimiento no depende de qué calcula la función, sino de que esté **definida por casos sobre la forma de su entrada**. Mientras cada regla diga qué hacer con una variante, y las partes se resuelvan con la misma función, el trazado siempre es el mismo movimiento: buscar la regla que aplica, sustituir, y repetir.

Eso es lo que te va a servir en el integrador, donde la especificación tiene un parámetro más y una regla que no habías visto.